MOBILE APPLICATION DEVELOPMENT

Taskaya

Phase 2 Report

A Flutter to-do application backed by Firebase Authentication and Cloud Firestore, with per-user task isolation, search and filtering, scheduled reminders, and a built-in Focus Mode.

TEAM

No.	Student Name	Student ID
1 (Team Leader)	Ahmed Sameh	
2	Mostafa	

Project Title: To-Do List Mobile Application
Phase 2 of 2 · Flutter · Firebase

1. Project Overview

Taskaya is a minimal, monochrome to-do application built with Flutter. Phase 1 delivered the splash screen, task list, add/edit/delete, completion toggling, navigation and an organized UI, all backed by local mock storage. Phase 2 replaces that storage with a real backend and adds the capabilities a multi-user cloud application requires.

Specifically, Phase 2 integrates **Firebase Authentication** (email/password, plus Google Sign-In) and **Cloud Firestore**, scoping every task to its owner so a signed-in user can only ever reach their own data. On top of that it adds task search, Completed/Pending filtering, hardened input validation, friendly error handling for both auth and network failures, and a round of UI/UX improvements. Task priority and due date/time carry forward from Phase 1 and are now persisted in Firestore.

For the bonus criteria the app ships dark mode, task categories, real local notifications with task reminders, animations throughout, and one additional creative feature: **Focus Mode**, a countdown workspace that ends in a celebration and marks the task complete.

Build status: `flutter analyze` reports **"No issues found!"** — zero errors, warnings or infos. The release APK builds successfully, and every feature described here was verified running on an Android emulator against the live `taskaya-1` Firebase project. All screenshots in section 8 are real captures from that session.

2. Requirements Coverage

Phase 2 requirements (10 marks)

Requirement	Status	Where it lives
Integrate Firebase Cloud Firestore	Done	firestore_task_repository.dart · §3.2
Firebase Authentication (Email & Password)	Done	firebase_auth_repository.dart · §3.1
Each user accesses only their own tasks	Done	firestore.rules, deployed · §3.3
Search tasks	Done	home_screen.dart, task_provider.dart · §5
Filter tasks (Completed / Pending)	Done	home_screen.dart · §5
Task Priority (High / Medium / Low)	Done	task_priority.dart · §4
Due Date & Time	Done	add_edit_task_screen.dart · §4
Input Validation	Done	validators.dart · §7
Improve UI/UX	Done	Toasts, animations, overdue flags · §7
Proper Error Handling	Done	Auth + Firestore error mapping · §3.4

2. Requirements Coverage (continued)

Bonus criteria (2 marks)

Bonus item	Status	Notes
Dark Mode	Done	Full dark theme, toggled from the header
Task Categories	Done	Personal / Study / Work / Other, with filters
Local Notifications	Done	Real scheduled OS notifications
Task Reminder	Done	User-chosen lead time before the due date
Animations	Done	Screen, list, checkbox, celebration
Additional creative feature	Done	Focus Mode with countdown + celebration
Push Notifications	Not implemented	Local notifications used instead
Voice Input	Not implemented	Placeholder replaced by Focus Mode

Deliverables

Deliverable	Status
Updated Flutter project	Included
APK file (release build)	Built
PDF report (this document)	Included

3. Firebase Integration

3.1 Authentication

A new **FirebaseAuthRepository** implements the app's existing **AuthRepository** interface, so no screen code changed to adopt it — only the dependency wiring in `main.dart`. Signup calls `createUserWithEmailAndPassword` and stores the display name; login calls `signInWithEmailAndPassword`; logout calls `signOut`. Session restore on the splash screen awaits the first `authStateChanges()` event rather than reading `currentUser` synchronously, avoiding a race with Firebase's asynchronous session restore on cold start.

Google Sign-In was added as a second provider. The provider was enabled on the project, an OAuth client created, and the debug keystore's SHA-1 and SHA-256 fingerprints registered against the Android app. `loginWithGoogle()` exchanges the returned Google ID token for a Firebase credential; signing out clears the Google session too, so the account picker reappears next time.

3.2 Cloud Firestore

Each user's tasks live at **users/{uid}/tasks/{taskId}**. `TaskProvider` subscribes to a live Firestore stream, so any change — from this device or another session — reflects immediately without a manual refresh. Every field is preserved: title, description, priority, category, due date/time, reminder offset, completion state, created date and completed date. Dates are stored as Firestore `Timestamp`s.

3.3 Security Rules & Per-User Isolation

Every task document is restricted to its owning user by rules deployed to the live project:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/tasks/{taskId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
    match /{document=**} { allow read, write: if false; }
  }
}
```

An unauthenticated request, or one for a uid that doesn't match the caller's own `auth.uid`, is denied outright — the trailing catch-all permits no other path. Because this is enforced server-side, the guarantee holds even if the client were modified.

3.4 Error Handling & Reliability

Firebase Auth error codes (invalid-email, wrong-password / user-not-found / invalid-credential, email-already-in-use, weak-password, network-request-failed, too-many-requests, user-disabled, internal-error) are mapped to short, actionable messages instead of raw exception text. Firestore failures are mapped similarly (permission-denied, offline/unavailable) and surfaced through `TaskProvider`'s error state as a dismissible banner rather than a silent failure or an endless loader. Login, signup, task save and task delete each guard against duplicate submissions while a request is in flight, disabling the button and showing a spinner for the duration.

4. Task Management: Priority & Due Date/Time

Every task carries a **priority** of High, Medium or Low, chosen from a three-way chip picker on the Add/Edit screen. Priority drives the ordering of the pending list — higher priority first, then earliest due date — and non-default priorities are shown inline on the task card.

Tasks can also carry a **due date and time**, set through separate date and time pickers themed to match the app, with a one-tap clear action. The due date drives three things: the relative label on the card ("Tomorrow, 00:36"), the overdue and due-soon warning states, and the scheduling of the task's reminder notification. All of it round-trips through Firestore as a proper timestamp.

5. Search, Filtering & Categories

A search field above the task list matches title and description, case-insensitively, across pending and completed tasks alike. An All / Pending / Completed chip row controls which sections render, and a second horizontally-scrollable row filters by category — Personal, Study, Work or Other — with a live task count on each chip. Selecting a category filters the list; tapping it again clears it.

All three controls compose, so a user can search within a single category while showing only pending items. When a combination matches nothing, a dedicated "No matching tasks" state appears with a one-tap reset — deliberately distinct from the empty state shown when the account genuinely has no tasks yet. Search and filtering run client-side over the already-streamed Firestore data, so they never re-issue a query and don't disturb the list animations.

6. Notifications & Reminders

Reminders are **real operating-system notifications**, not just in-app messages. They are scheduled with `zonedSchedule` using `exactAllowWhileIdle` and the device's true IANA timezone, resolved at runtime — so they fire even when the app is closed.

When creating a task the user chooses **how far in advance** to be reminded: No reminder, At due time, or 10 minutes / 30 minutes / 1 hour / 3 hours / 1 day before. The picker stays disabled with an explanatory hint until a due date is set, then enables with "At due time" preselected. The service schedules at `dueDate − offset` and writes a context-aware body. Editing, completing or deleting a task cancels and reschedules its reminder so notifications never go stale, and tapping one opens that exact task.

Reminders are isolated per account. Signing out cancels every scheduled notification, and the first Firestore snapshot for a newly signed-in user re-arms reminders from scratch — so on a shared device one account can never receive another's reminders. The in-app notification bell from Phase 1 is untouched and still works as a complementary inbox regardless of OS permission state.

7. Focus Mode (creative feature)

Focus Mode is the additional creative feature, aimed at actually *doing* the work rather than only listing it. From the Focus button on Home — or "Focus on this task" in a task's long-press menu — the user picks a duration and a pending task, then enters a full-screen countdown with an animated progress ring, a slow breathing pulse, pause/resume, and an "I finished early" shortcut. The task picker reads an unfiltered pending list, so a narrowed Home view never hides tasks from it.

When the timer ends a real OS notification fires (the app is usually backgrounded by then) and the user is asked whether they finished. Answering yes marks the task complete in Firestore and plays a full-screen celebration; answering no offers +5, +10 or +15 more minutes, or an exit. The celebration is built entirely from `CustomPainter` and staggered animations rather than an imported animation asset — ballistic confetti from a seeded RNG, a spring badge pop, a checkmark that draws itself, and copy that slides up — keeping it monochrome to match the app and collapsing to a static state under reduced-motion.

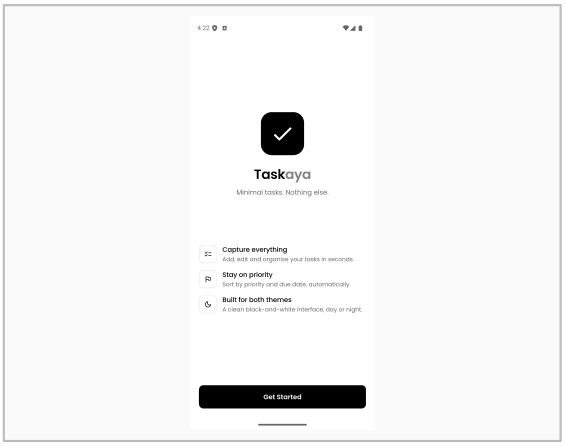
8. Input Validation & UI/UX Improvements

Passwords created at signup must be at least 8 characters and contain both a letter and a number, shown as a live hint under the field and validated as the user types. Login deliberately keeps a presence-only check: applying the new rule there would lock out any account created under the earlier 6-character minimum. Validation also covers a stricter email pattern and length cap, name length bounds, an empty confirm-password field, task title length, description length (500 characters) and whitespace-only passwords.

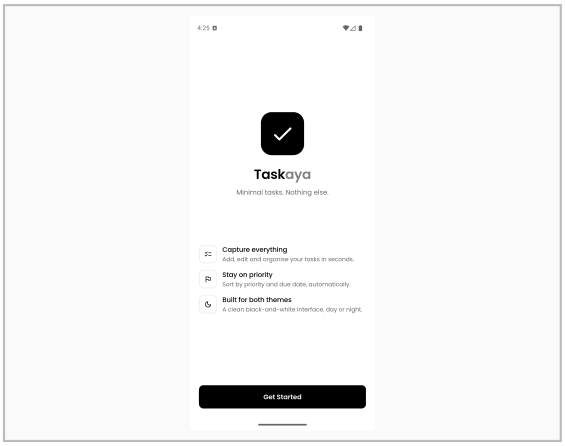
On the UI/UX side, every task action confirms itself with an icon-led floating toast — "Task "…" created", "Task updated", "Task completed", "Task deleted" with an Undo action, and error variants. Completing a task emits an expanding ring burst from the checkbox (only on unchecked → checked, so undoing doesn't celebrate). Task cards flag **Overdue** with a red border, alert icon and bold label, or **due within the hour** with a stronger border and clock icon. Dark mode and reduced-motion support carry over from Phase 1 and apply to all of the new screens.

9. Screenshots

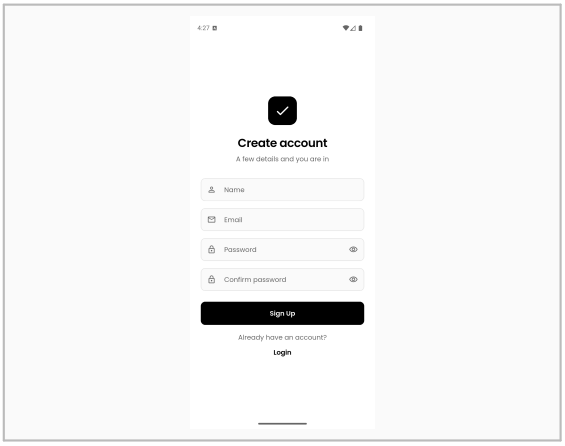
All captures are from a live Android emulator (Pixel 9a) running a debug build signed in to the deployed `taskaya-1` Firebase project, against real Firestore data.



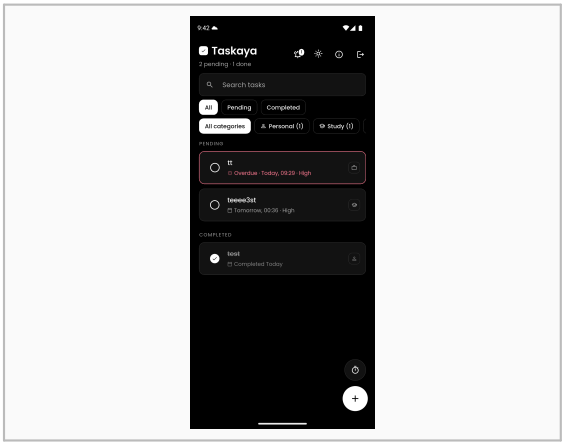
Onboarding — first-run screen



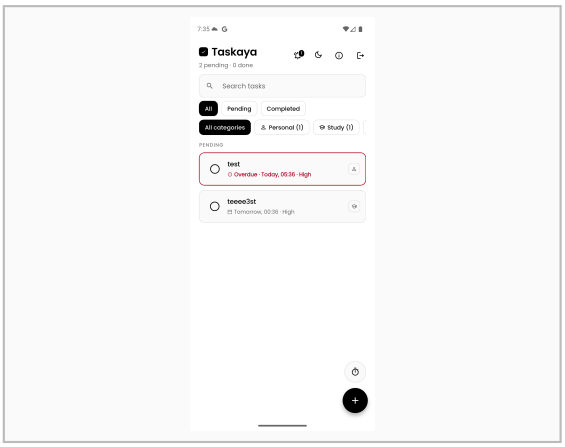
Login — email/password and Google



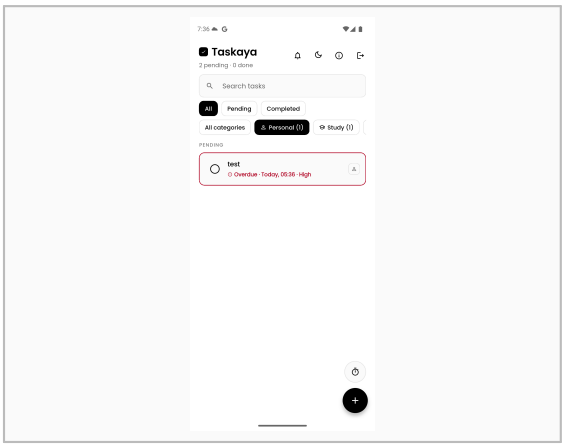
Signup — with password requirements



Dark mode (bonus) — full task list

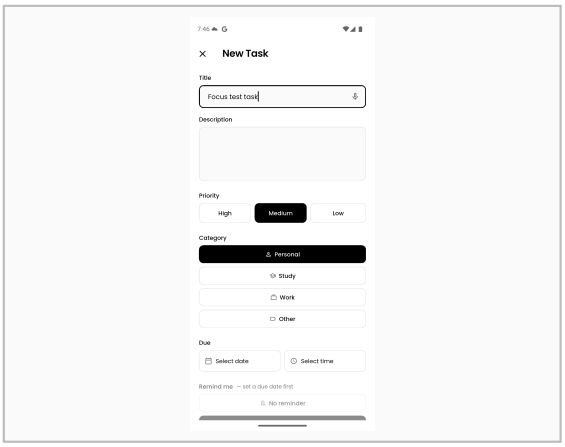


Home — Firestore tasks, categories, overdue flag

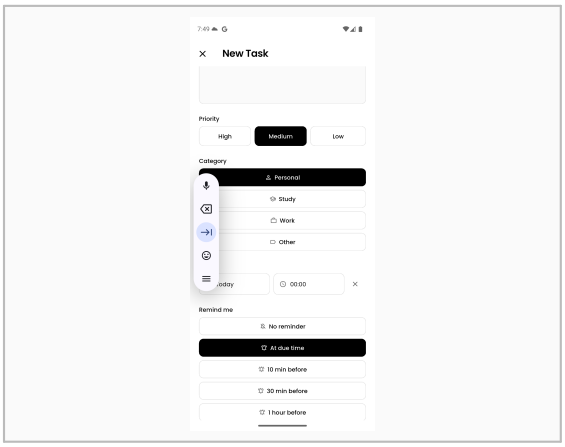


Category filter applied (Personal)

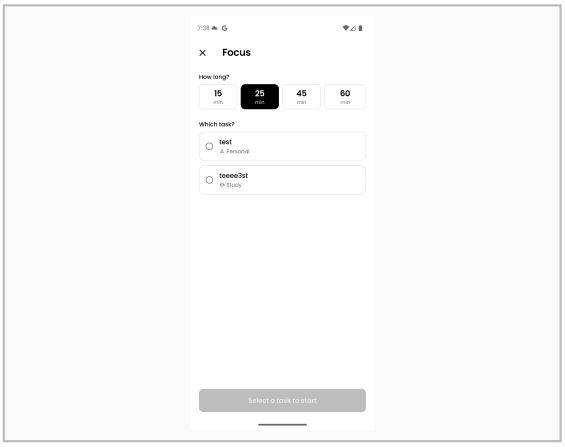
9. Screenshots (continued)



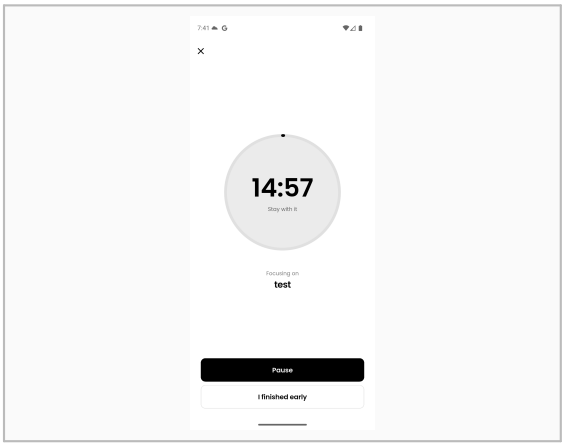
Add task — priority and category pickers



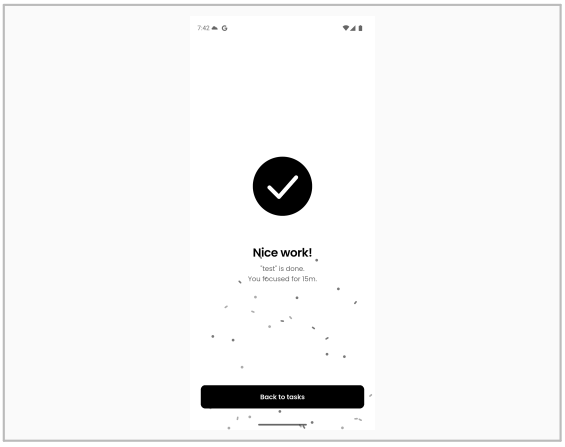
Reminder lead-time picker, enabled by a due date



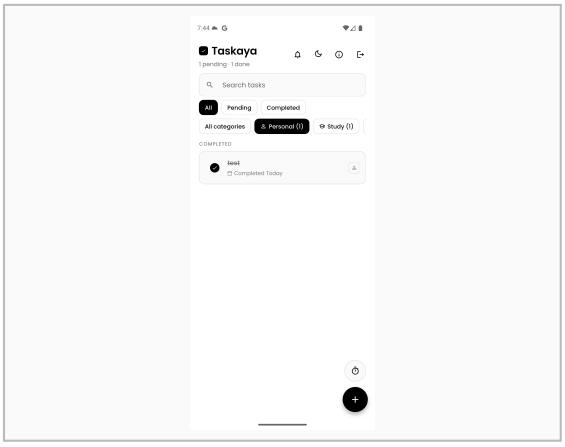
Focus Mode — pick duration and task



Focus countdown with progress ring



Celebration after completing a focus session



Task marked complete and synced to Firestore

10. Team Contributions

Area	Owner	Outcome
Firebase setup, Authentication, notifications	Mostafa	Real accounts, Google Sign-In, secure session handling, scheduled device rer
Firestore tasks, search/filter, categories	Ahmed	Cloud-synced, user-scoped task management and discovery controls
Focus Mode, animations, feedback	Shared	Countdown workspace, celebration reward, toasts and lifecycle animations
Security rules, QA, APK and report	Shared	A safe, demonstrable, submittable Phase 2 release

11. Challenges & Decisions

- The original **TaskRepository** was a concrete, synchronous, in-memory class with no interface. It was turned into an abstract interface — mirroring the existing AuthRepository pattern — so TaskProvider could become stream-driven and Firestore-backed without disturbing any screen code.
- Session restore raced Firebase Auth's asynchronous persistence check on cold start: reading `currentUser` synchronously could momentarily return null even with a valid stored session. Awaiting the first `authStateChanges()` event resolves it correctly.
- The release build failed with a non-obvious Gradle error: `flutter_local_notifications` requires Android core library desugaring, which was not enabled. Adding `isCoreLibraryDesugaringEnabled` and the `desugar_jdk_libs` dependency fixed it.
- Accurate scheduling needs the device's real IANA timezone, not just a UTC offset, so `flutter_timezone` was added alongside the notification and timezone packages to resolve it at runtime.
- Making all task mutations asynchronous (Firestore round-trips instead of instant local writes) meant every call site needed a loading state, a duplicate-submission guard, and a way to surface a network error without crashing or hanging.
- Email/password signup failed on the test device with `RecaptchaCallWrapper … [ CONFIGURATION_NOT_FOUND ]`. Logs confirmed the app was calling the API correctly — recent Firebase Auth SDKs run a reCAPTCHA Enterprise check on sign-up, and that key had not finished provisioning for this newly created project. Google Sign-In was added as the practical unblock and is how the app was verified end-to-end; the email/password path is implemented and expected to work once provisioning completes.
- Rather than add a Lottie or Rive dependency for the celebration, it was hand-built with CustomPainter and staggered animations. This keeps the app dependency-light, matches the strict monochrome design language, and made reduced-motion support straightforward.

12. Conclusion

All ten Phase 2 requirements are implemented and verified: Firestore integration, Firebase Authentication, per-user task isolation, search, Completed/Pending filtering, task priority, due date and time, input validation, UI/UX improvements and proper error handling. Six of the eight bonus items are delivered,

including a creative feature of our own in Focus Mode.

A real account can sign in, create, search, filter, update and delete tasks that persist in Firestore, and cannot access another user's data. The app handles invalid and offline states cleanly and schedules real device reminders at a user-chosen lead time. `flutter analyze` reports no issues and the project compiles and runs without errors.